

Apuntes de clase del 28/08/26: Proyecto 1

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Redes - IC7602
II Semestre de 2026
Sebastián Calvo Hernández - 2022099320

Introducción

En esta clase prácticamente todo se dedicó a explicar el proyecto 1. La idea es trabajar con servicios de capa de aplicación y ver cómo se apoyan en protocolos de transporte como UDP y TCP.

El proyecto vale 20%, se realiza en grupos de máximo 5 personas y al final hay una evaluación presencial escrita. El profesor recalcó que todos los integrantes deben entender la implementación y no solamente tener una parte asignada.

El proyecto mezcla varios temas:

- DNS
- UDP y TCP
- HTTP/HTTPS
- Base64
- API REST
- Docker
- Supabase o Firebase
- Health checks
- Automatización

La idea es hacer una versión pequeña de un servicio DNS para entender cómo se comunican distintos componentes y cómo se toman decisiones sobre qué IP devolver.

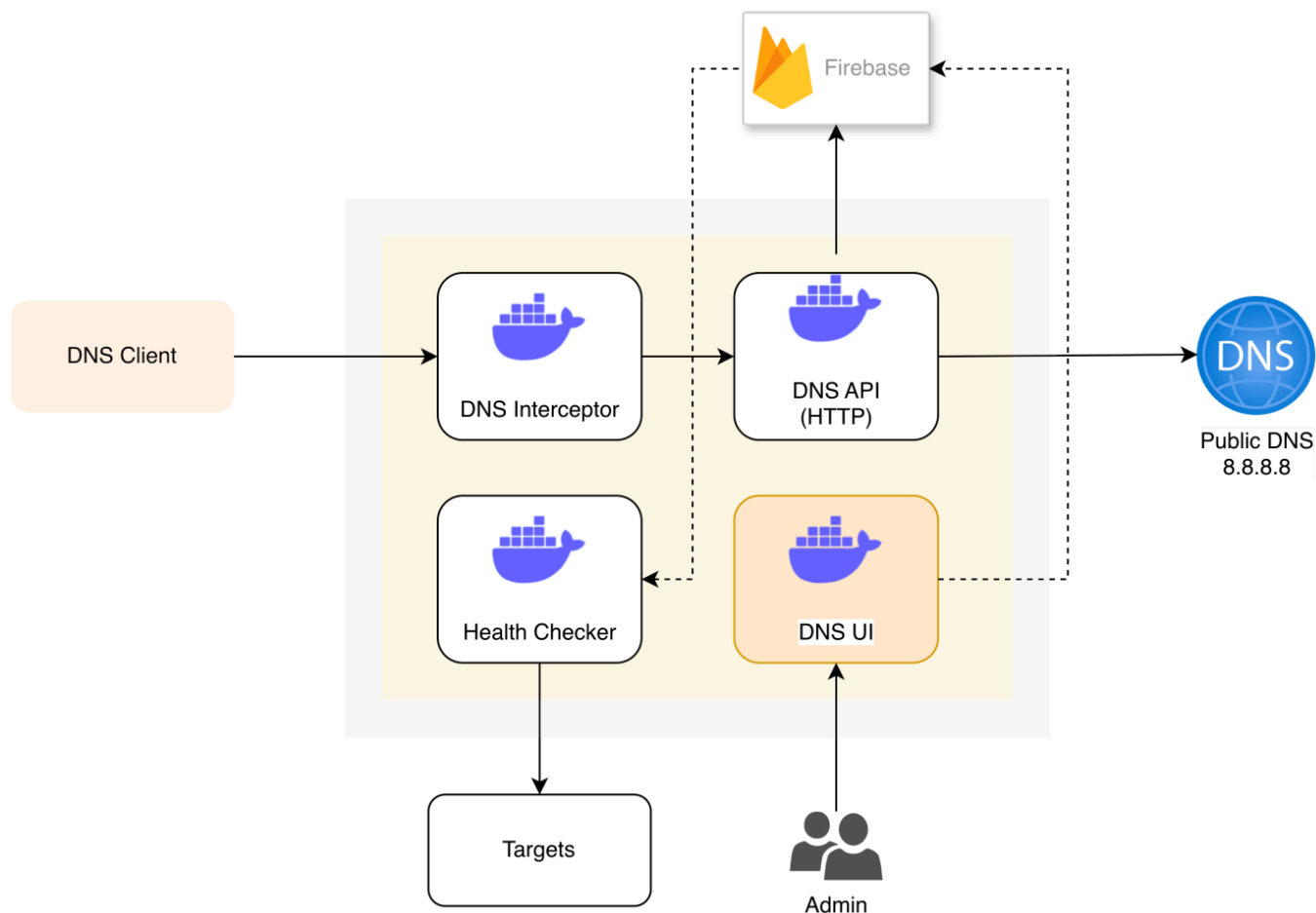
Funcionamiento básico de DNS

Cuando escribimos un nombre como `www.ejemplo.com` la computadora necesita obtener primero la dirección IP correspondiente.

Para esto se utiliza un resolver. El resolver puede revisar si ya tiene la respuesta en caché y, si no la tiene, debe consultar a un servidor DNS.

DNS normalmente trabaja sobre UDP/53. UDP es un protocolo no orientado a conexión y no ofrece las mismas garantías que TCP.

Una vez que la computadora consigue la IP, ya puede continuar con las conexiones necesarias para solicitar la información del servicio.



DNS Interceptor

El primer componente es el DNS Interceptor, debe ejecutarse en un contenedor Docker, estar escrito en Rust y escuchar en:

UDP/53

La función del interceptor es recibir las consultas DNS antes de que lleguen directamente a un DNS externo. La idea es que esto sea transparente para el cliente, si se implementa correctamente, herramientas como nslookup deberían poder utilizarlo sin problemas.

QR y OPCODE

El interceptor tiene que revisar directamente el encabezado del paquete DNS. Para identificar un query estándar se utilizan:

QR = 0

OPCODE = 0

El bit QR permite saber si el paquete corresponde a una consulta o a una respuesta. El OPCODE permite identificar el tipo de operación.

Como el mensaje DNS es binario, hay que trabajar con bytes y bits. En clase el profesor mostró un ejemplo donde utilizaba desplazamientos de bits para extraer específicamente QR y OPCODE. Lo importante es

respetar las posiciones indicadas por los RFC, ya que mover o interpretar un bit incorrectamente puede hacer que el paquete deje de ser válido.

Paquete que NO es un query estándar

Si el paquete recibido no cumple:

QR = 0 OPCODE = 0

el Interceptor debe:

1. Tomar el paquete DNS.
2. Codificarlo en Base64.
3. Enviarlo por HTTPS POST al DNS API.
4. Recibir la respuesta en Base64.
5. Decodificarla.
6. Enviar el paquete DNS al cliente.

Base64 se utiliza porque el paquete DNS contiene información binaria, mientras que entre el Interceptor y el API se está transportando esa información mediante HTTP/HTTPS.

Aquí se puede ver una de las ideas principales del proyecto:

DNS/UDP -> Base64 -> HTTP/HTTPS -> TCP

Es decir, estamos transportando información de un protocolo que trabaja sobre UDP a través de otro servicio que trabaja sobre TCP.

Query estándar y labels de DNS

Si el paquete sí tiene:

QR = 0 OPCODE = 0

entonces hay que identificar el hostname que se está tratando de resolver.

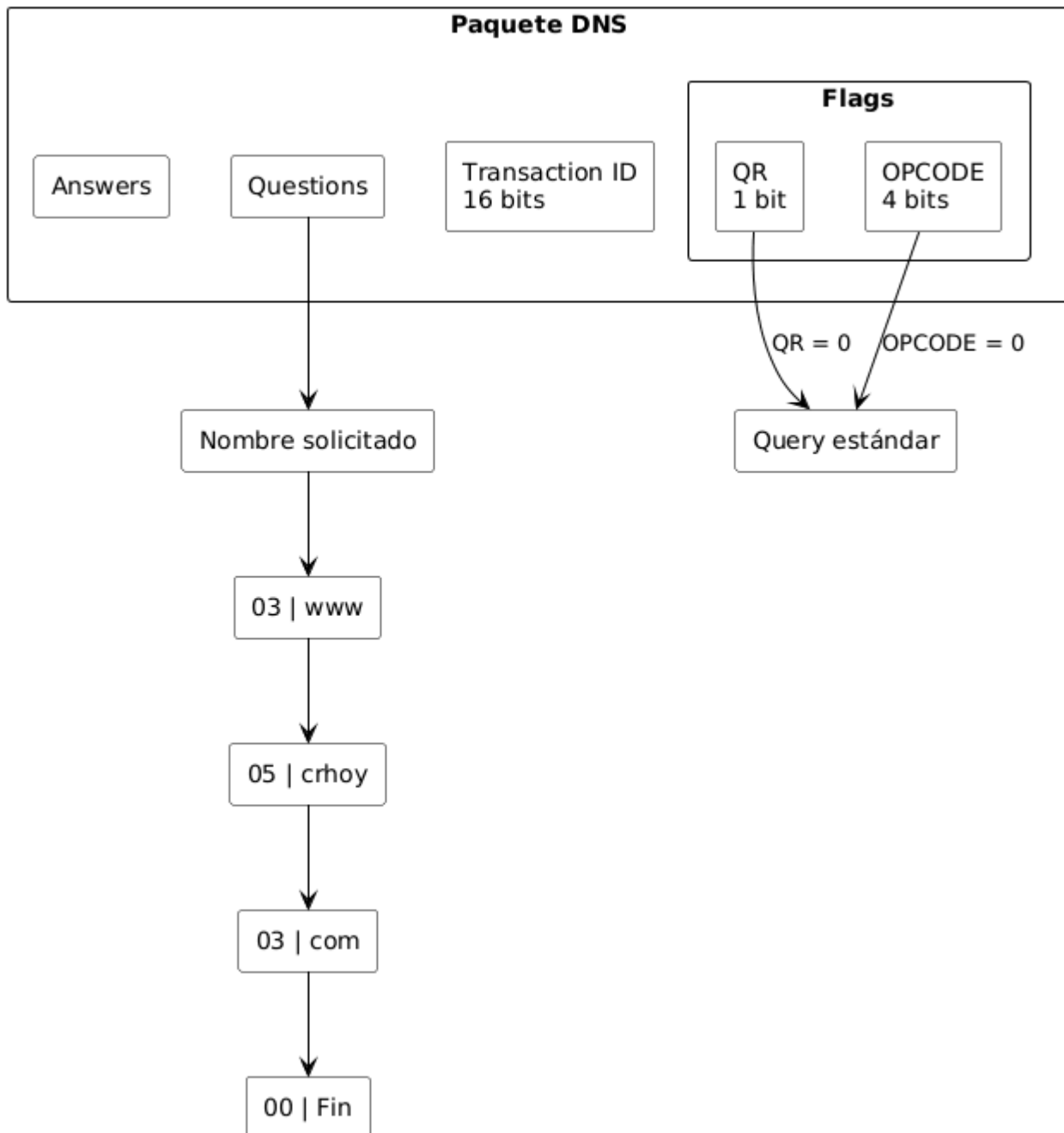
En clase se utilizó un ejemplo de www.crhoy.com

En un paquete DNS el nombre se representa mediante labels:

www crhoy com

Antes de cada label aparece un valor que indica cuántos caracteres tiene. Por ejemplo, un 03 indica que vienen tres caracteres.

El programa tiene que recorrer esa información y reconstruir el nombre del host para después preguntarle al DNS API si existe un registro en Supabase o Firebase.



DNS API

El DNS API debe ejecutarse en otro contenedor y estar implementado en Java como una API REST. Como mínimo debe tener: `/api/exists` y `/api/dns_resolver`

`/api/exists`

Sirve para verificar si un dominio existe dentro de Supabase o Firebase.

`/api/dns_resolver`

Recibe el paquete DNS codificado en Base64, lo decodifica y lo envía a un servidor DNS externo mediante UDP, después recibe la respuesta, vuelve a codificarla en Base64 y se la devuelve al DNS Interceptor. El API

debe implementar entonces un cliente UDP/DNS, la dirección del DNS externo tiene que llegar por configuración.

En clase se utilizó como ejemplo:

8.8.8.8

También debe soportar múltiples peticiones de manera concurrente.

Tipos de registros

La base de datos puede contener varios tipos de registros.

Single

Un dominio tiene una única IP y simplemente se retorna ese valor.

dominio -> IP

Multi

Un dominio tiene varias IPs y se utiliza round-robin.

La idea es ir alternando:

Petición 1 -> IP 1 Petición 2 -> IP 2 Petición 3 -> IP 3 Petición 4 -> IP 1

Weight

También existen varias IPs, pero cada una tiene un peso.

Ejemplo:

IP 1 -> 50 IP 2 -> 20 IP 3 -> 30

La IP con mayor peso debería recibir más peticiones.

Geo

La IP que se devuelve depende del país desde donde viene el cliente.

Se toma la IP de origen y se consulta una base de datos IP to Country. Con esto se obtiene el país y se devuelve la IP configurada para esa ubicación. Si no hay una IP definida para ese país, se devuelve una de forma aleatoria.

Round-trip

Este registro utiliza la información generada por los Health Checkers. La idea es devolver una IP tomando en cuenta la cercanía de la persona usuaria y cuál servidor tiene menor latencia.

Para el proyecto las ubicaciones pueden ser simuladas y también se permite simular o aplicar penalizaciones a los valores de round-trip para poder demostrar el funcionamiento.

Health Checker

El Health Checker debe estar escrito en GNU C y ejecutarse en un contenedor Docker.

Su función es revisar si los servidores configurados siguen disponibles.

Se deben manejar dos tipos de prueba:

TCP

Se intenta establecer una conexión a un puerto. Si la conexión puede abrirse, se considera que el servicio responde.

HTTP

Se realiza una petición HTTP y se revisa el resultado.

Se pueden configurar:

- Path.
- Timeout.
- Retries.
- Intervalo entre pruebas.
- Código HTTP esperado.

Por ejemplo:

HTTP 200 OK

Los Health Checkers marcan los registros como:

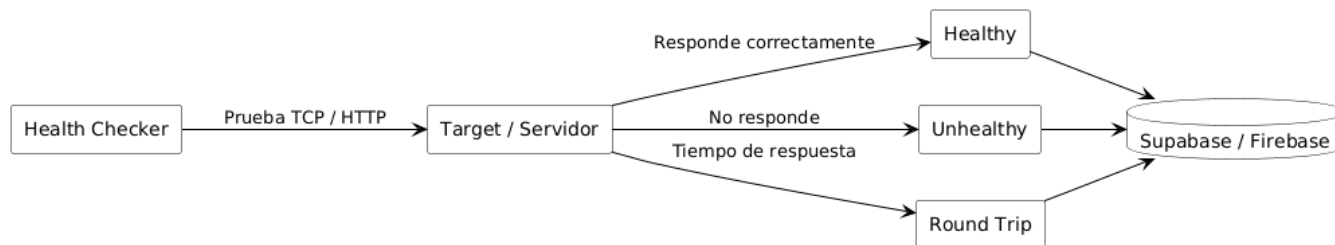
healthy

o:

unhealthy

Cuando existen varios Health Checkers se utiliza mayoría simple para decidir el estado final, también miden el round-trip time, que luego se utiliza para los registros tipo round-trip.

Cada Health Checker puede tener una ubicación geográfica simulada con latitud, longitud, país y ciudad.



DNS UI

El DNS UI es una aplicación en React que sirve para administrar la configuración.

Debe permitir crear, modificar y eliminar registros de tipo:

- single
- multi
- weight
- round-trip
- geo

También se configuran los Health Checks y se administra la información de la base de datos IP to Country.

Para esta parte sí se permite IA generativa, pero deben documentarse los prompts utilizados y los problemas encontrados.

Automatización

Todo el proyecto debe quedar automatizado.

Se pueden utilizar herramientas como:

- Docker.
- Docker Compose.
- Kubernetes.
- Helm Charts.

No se deben dejar valores de configuración quemados directamente en el código. La configuración se debe pasar mediante variables de entorno.

Este punto es bastante importante porque si la instalación no está automatizada, el proyecto puede recibir una nota de 0.

Pruebas recomendadas

Para probar el funcionamiento de DNS se recomendó utilizar:

nslookup

Para interactuar con el DNS API se puede utilizar Postman.

Una prueba más completa sería modificar el resolv.conf de la computadora para utilizar como DNS la IP del contenedor. Si después de eso la resolución de nombres continúa funcionando correctamente, sería una buena prueba de que el Interceptor está trabajando como se espera.

Documentación y entrega

Toda la documentación debe hacerse en Markdown.

Debe incluir:

- Instrucciones para ejecutar el proyecto.
- Pruebas y pasos para reproducirlas.
- Al menos 10 conclusiones.
- Al menos 10 recomendaciones.
- Tabla indicando qué funciona y qué no funciona.

También hay que entregar reportes de avance todos los jueves con las tareas, responsables, avances, problemas, commits y Pull Requests.

Evaluación

Rubro	Valor
Documentación	10%
DNS Interceptor	10%
DNS API	5%
DNS UI	5%
Health Checker	5%
Reportes y GitHub	5%
Evaluación presencial	60%

Fechas

- Entrega: 22/09/2026 antes de medianoche.
- Evaluación presencial: 29/09/2026.
- Reportes de avance: todos los jueves antes de medianoche.